# Unsupervised Learning: Dimensionality Reduction & Clustering

## STAT3612: Statistical Machine Learning

Yinghao Zhu

HKU

November 2026

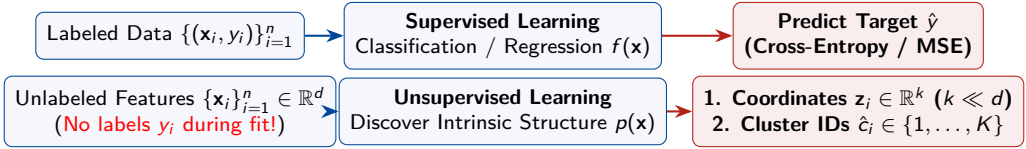
# Outline & Companion Notebook

1. Part 1: Dimensionality Reduction (PCA, Kernel PCA & t-SNE)
2. Part 2: Clustering (K-Means, Hierarchical, DBSCAN & GMM)
3. Part 3: Unsupervised Pipelines in Practice

## Hands-On Companion: `unsupervised_learning_tutorial.ipynb`

- **Part 1 (Dimensionality Reduction):** `StandardScaler` + `PCA` geometry, scree plots (`explained_variance_ratio_`), `PCA` vs. `LDA`, `KernelPCA`, and `TSNE`.
- **Part 2 (Clustering):** `KMeans` alternating optimization, elbow & `silhouette_score`, non-convex failure modes, `AgglomerativeClustering` dendrograms, and `GaussianMixture`.
- **Part 3 (Unsupervised Pipelines in Practice):** `PCA` denoising before `KNeighborsClassifier` and combining `PCA` + `KMeans` on high-dimensional data.

# Quick Design Recap: Supervised vs. Unsupervised Learning




## 1. Dimensionality Reduction ($\mathbb{R}^d \to \mathbb{R}^k$)

- Compress correlated features into $k \ll d$ orthogonal axes (`PCA`, `KernelPCA`) or low-D manifolds (`TSNE`).
- Mitigates the curse of dimensionality, denoises distance metrics (like KNN), and enables 2D/3D visualization.

## 2. Clustering ($\mathbf{x}_i \mapsto \hat{c}_i \in \{1, \ldots, K\}$)

- Partition unlabeled samples into homogeneous groups (`KMeans`, `AgglomerativeClustering`, `DBSCAN`, `GaussianMixture`).
- Discovers latent subtypes, customer/student personas, and outlier anomalies.

# Visual Intuition: Principal Component Analysis (PCA) Geometry

**Two Equivalent Views of PCA:**

- **Max Variance View:** Find unit axis $\mathbf{w}_1$ ($\|\mathbf{w}_1\|_2 = 1$) maximizing projected variance:

$$\max_{\|\mathbf{w}_1\|=1} \frac{1}{n}\sum_{i=1}^{n}(\mathbf{w}_1^\top \tilde{\mathbf{x}}_i)^2 = \mathbf{w}_1^\top \mathbf{S}\mathbf{w}_1$$

- **Min Reconstruction View:** Minimize orthogonal squared error $\sum_i \|\tilde{\mathbf{x}}_i - (\mathbf{w}_1^\top \tilde{\mathbf{x}}_i)\mathbf{w}_1\|_2^2$.
- **Eigendecomposition / SVD:** $\mathbf{w}_1, \ldots, \mathbf{w}_k$ are the top-$k$ eigenvectors of sample covariance $\mathbf{S} = \frac{1}{n}\tilde{\mathbf{X}}^\top\tilde{\mathbf{X}}$ with eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d$.

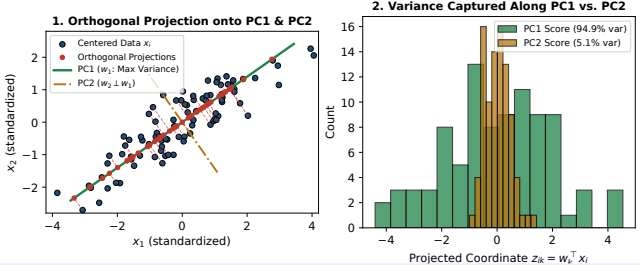



**Figure Takeaway:** **PC1** ($\mathbf{w}_1$, green) aligns with the direction of widest spread (capturing 93.4% of variance) while minimizing the red dashed orthogonal residuals; **PC2** ($\mathbf{w}_2 \perp \mathbf{w}_1$, gold) captures the remaining orthogonal variance.

# Dataset & Task Roadmap: Exploring the 4D Iris Feature Space

**What Is the Dataset & Goal?**

- **Dataset:** `load_iris()` with $n = 150$ flowers across 3 species (*Setosa*, *Versicolor*, *Virginica*).
- **Features ($d = 4$):** `sepal length`, `sepal width`, `petal length`, and `petal width` (in cm).
- **Observation:** Sepal measurements overlap heavily, whereas petal length and petal width are strongly collinear ($r = 0.96$).
- **Goal:** Compress 4D $\rightarrow$ 2D/3D without using species labels $y$.

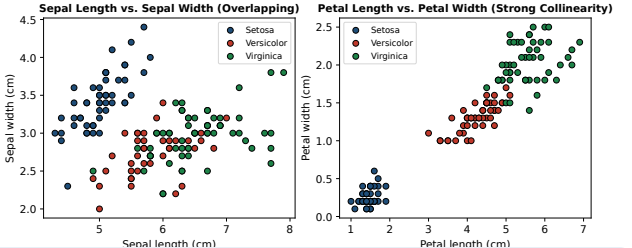



**Figure Takeaway:** Because `petal length` and `petal width` lie nearly on a 1D line, the 4D feature space has strong redundancy that linear PCA can compress into just 2 principal components.

# Step 1: Why StandardScaler Before PCA Is Mandatory

```python
from sklearn.preprocessing import
StandardScaler
from sklearn.decomposition import PCA
# Wrong:  raw features with unequal units!
pca_raw = PCA(n_components=2).fit(X_wine)
# Correct:  standardize to zero mean, unit var
scaler = StandardScaler()
X_std = scaler.fit_transform(X_wine)
pca_std = PCA(n_components=2).fit(X_std)
Z_std = pca_std.transform(X_std)
```

- **Why Scale?** PCA maximizes variance $\mathbf{w}^\top \mathbf{S}\mathbf{w}$. Without scaling, a feature measured in large numeric units (e.g., proline $\in [278, 1680]$ in Wine) hijacks PC1 (99.8% variance!).
- **Takeaway:** Always apply `StandardScaler()` before `PCA` (and fit the scaler on `X_train` only).

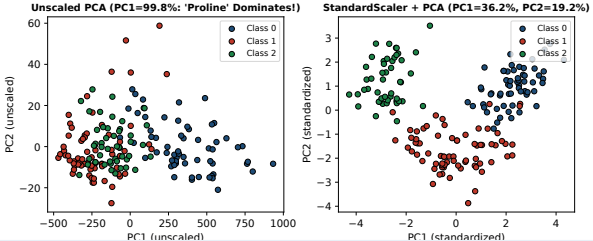



**Figure Takeaway:** On the 13-feature Wine dataset, unscaled PCA collapses onto a single large-scale feature (PC1 = 99.8%, mixed classes), whereas `StandardScaler + PCA` cleanly separates all 3 classes.

# Step 2: Scree Plot & Cumulative Explained Variance Ratio

```
pca = PCA().fit(X_std)
evr = pca.explained_variance_ratio_
cum_evr = np.cumsum(evr)
# Or let sklearn pick smallest k >= 95% var:
pca_95 = PCA(n_components=0.95).fit(X_std)
print(pca_95.n_components_)
```

- **Explained Variance Ratio (EVR):**

$$\text{EVR}_k = \frac{\lambda_k}{\sum_{j=1}^{d} \lambda_j}, \quad \text{CumEVR}_k = \sum_{m=1}^{k} \text{EVR}_m$$

- **How to Choose $k$:** Plot $\text{EVR}_k$ (bars) and $\text{CumEVR}_k$ (step/line curve); pick the scree elbow or the smallest $k$ crossing 80%–95% variance.

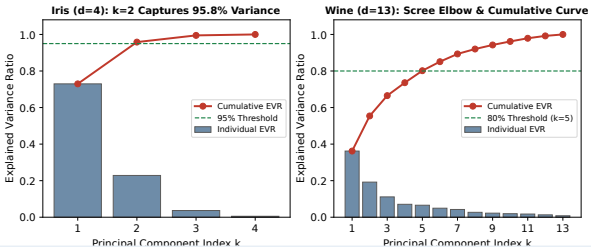
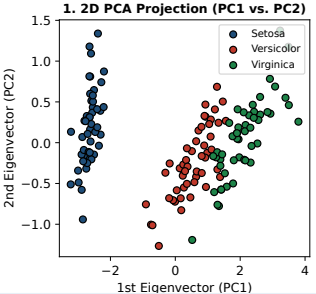



**Figure Takeaway:** On Iris ($d = 4$), $k = 2$ components retain 95.8% of total variance; on Wine ($d = 13$), the scree plot shows a gradual decay reaching 80% variance at $k = 5$ components.

# Step 3: 2D & 3D PCA Projections of the Iris Dataset

```
# Project 4D Iris -> 3D principal subspace
X_reduced =
PCA(n_components=3).fit_transform(
  iris.data)
# 2D scatter uses columns 0 and 1
ax2d.scatter(X_reduced[:, 0], X_reduced[:,
1],
  c=iris.target)
# 3D scatter uses columns 0, 1, and 2
ax3d.scatter(X_reduced[:, 0], X_reduced[:,
1],
  X_reduced[:, 2], c=iris.target)
```

- **Takeaway:** Even though `PCA` never sees `iris.target`, projecting 4D measurements onto the top 2 or 3 eigenvectors reveals well-separated species clusters.



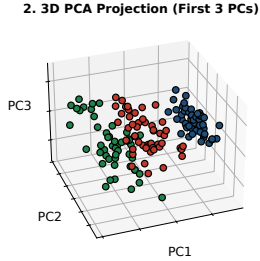




**Figure Takeaway:** Both the **2D PCA Projection** (PC1, PC2) and **3D PCA Projection** (PC1, PC2, PC3) isolate *Setosa* completely and separate *Versicolor* from *Virginica* along PC1.

# PCA vs. LDA: Unsupervised Variance vs. Supervised Separation

```
# Unsupervised:  fits on X only!
pca = PCA(n_components=2)
X_r = pca.fit(X).transform(X)
# Supervised:  fits on (X, y) to separate
classes
lda =
LinearDiscriminantAnalysis(n_components=2)
X_r2 = lda.fit(X, y).transform(X)
```

- **PCA (Unsupervised):** Maximizes total variance $\mathbf{w}^\top \mathbf{S}_T \mathbf{w}$ regardless of class labels.
- **LDA (Supervised):** Maximizes Fisher's ratio $\frac{\mathbf{w}^\top \mathbf{S}_B \mathbf{w}}{\mathbf{w}^\top \mathbf{S}_W \mathbf{w}}$ (between-class over within-class scatter) and has at most $\min(d, C-1)$ axes.

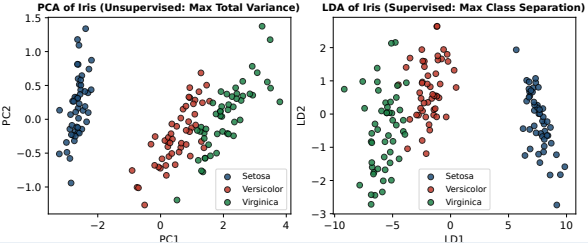



**Figure Takeaway:** Because `LDA` uses class labels $y$, its first discriminant axis (LD1) achieves tighter within-class compactness and wider separation between *Versicolor* (red) and *Virginica* (green) than unsupervised `PCA`.

# Non-Linear Dimensionality Reduction: `KernelPCA` on Concentric Rings

```python
from sklearn.decomposition import KernelPCA
# Linear PCA cannot unroll concentric circles
pca = PCA(n_components=2).fit(X_train)
X_pca = pca.transform(X_test)
# RBF KernelPCA maps to high-D RKHS
kpca = KernelPCA(kernel="rbf", gamma=10,
  fit_inverse_transform=True, alpha=0.1)
X_kpca = kpca.fit(X_train).transform(X_test)
```

- **How `KernelPCA` Works:** Performs eigendecomposition on the centered $n \times n$ kernel matrix $\mathbf{K}_{ij} = K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$ without computing $\phi(\mathbf{x})$ explicitly.

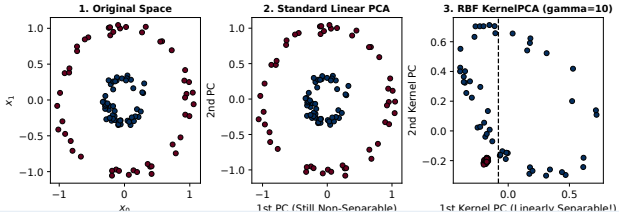



**Figure Takeaway:** Standard linear `PCA` only rotates the concentric rings (still non-separable), whereas RBF `KernelPCA(gamma=10)` makes the two rings **linearly separable** along the 1st Kernel PC!

# Manifold Learning for Visualization: PCA vs. TSNE on Digits

```python
from sklearn.manifold import TSNE
# 64-D handwritten digit images (8x8 pixels)
Z_pca =
PCA(n_components=2).fit_transform(X_dig)
# t-SNE matches pairwise neighbor affinities
tsne = TSNE(n_components=2, perplexity=30,
  init="pca", random_state=42)
Z_tsne = tsne.fit_transform(X_dig)
```

- **How t-SNE Works:** Converts Euclidean distances into conditional probabilities $p_{j|i}$ in 64D and minimizes KL divergence to a heavy-tailed Student-$t$ distribution $q_{ij}$ in 2D.
- **Caveat:** TSNE has no .transform(X_test) method—use it for **2D visualization**, not downstream feature pipelines!

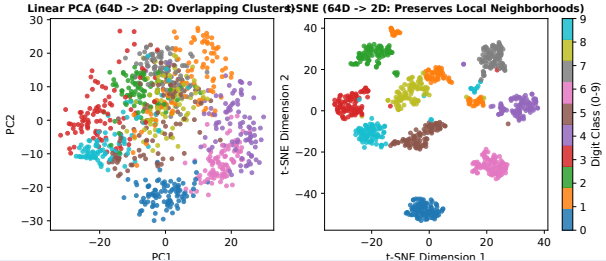



**Figure Takeaway:** On 64D handwritten digits (0–9), 2D linear PCA crowds digit classes in the center, whereas non-linear TSNE resolves 10 distinct local islands.

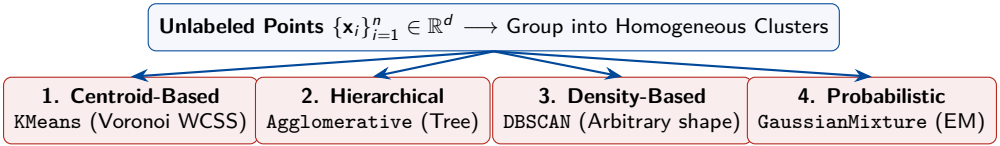
# Part 1 Summary: Dimensionality Reduction Method Comparison

| Method | Objective | Key `sklearn` Args | Uses $y$? | `.transform(X_te)`? |
|---|---|---|---|---|
| **Linear** `PCA` | Max global variance / min L2 error | `n_components` ($k$ or 0.95) | No | **Yes** ($O(dk)$) |
| `KernelPCA` | Max variance in RKHS $\phi(\mathbf{x})$ | `kernel="rbf"`, `gamma` | No | **Yes** ($O(nk)$) |
| `LDA` | Max between / within-class scatter | `n_components` ($\leq C-1$) | Yes | **Yes** ($O(dk)$) |
| `TSNE` | Match local neighborhood KL div. | `n_components=2`, `perplexity` | No | No (`fit_transform`) |

## Leak-Free Downstream Supervised Pipeline Pattern

```
pipe = make_pipeline(StandardScaler(), PCA(n_components=0.95), KNeighborsClassifier(n_neighbors=5))
pipe.fit(X_train, y_train)  # Fits scaler & PCA eigenvectors on X_train ONLY; transforms X_test cleanly!
```

# Quick Design Recap: Four Major Clustering Families




## Centroid & Probabilistic (KMeans, GMM)

- **K-Means:** Minimizes Within-Cluster Sum of Squares (WCSS / Inertia):

$$\min_{\{C_k, \boldsymbol{\mu}_k\}} \sum_{k=1}^{K} \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|_2^2$$

- **GMM:** Soft probabilistic generalization with full covariance matrices $\boldsymbol{\Sigma}_k$ fitted via EM.

## Hierarchical & Density (Agglomerative, DBSCAN)

- **Agglomerative:** Bottom-up tree merging pairs of clusters by linkage distance (`ward`, `complete`, `average`, `single`).
- **DBSCAN:** Groups dense $\varepsilon$-neighborhoods and flags isolated points as noise (`label = -1`) without pre-specifying $K$.

# Visual Intuition: How K-Means Alternates Assignment & Updates

**Lloyd's Alternating Algorithm:**

1. **Initialize:** Pick $K$ starting centroids $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_K$ (using `init="k-means++"` to spread them out).
2. **E-Step (Assign):** Assign each $\mathbf{x}_i$ to its nearest centroid:

$$\hat{c}_i = \arg \min_{k \in \{1,\ldots,K\}} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|_2^2$$

3. **M-Step (Update):** Recompute each centroid as the cluster mean $\boldsymbol{\mu}_k = \frac{1}{|C_k|} \sum_{\mathbf{x}_i \in C_k} \mathbf{x}_i$.
4. **Repeat** until assignments stop changing (guaranteed to monotonically decrease inertia!).

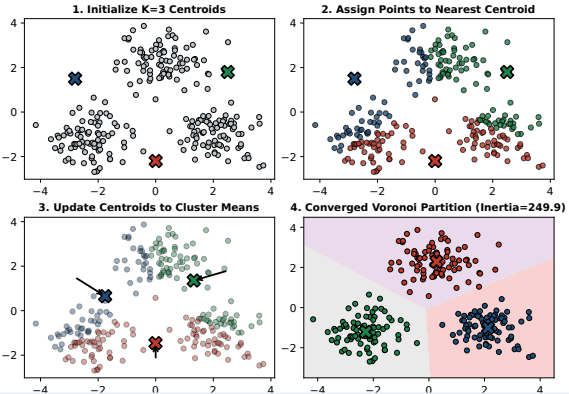



**Figure Takeaway:** K-Means alternates between assigning points to the closest centroid (panel 2) and shifting centroids to the cluster means (panel 3) until converging to a linear Voronoi partition (panel 4).

# Hands-On KMeans & Choosing $K$: Elbow vs. Silhouette Score

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
for k in range(2, 9):
  km = KMeans(n_clusters=k, init="k-means++",
    n_init=10, random_state=42).fit(X_std)
  inertias.append(km.inertia_)
  sils.append(silhouette_score(X_std,
km.labels_))
```

- **Inertia (`km.inertia_`):** Always decreases with $K$; look for the elbow where marginal gain flattens.
- **Silhouette Score** $s(i) = \frac{b(i)-a(i)}{\max(a(i),b(i))} \in [-1, 1]$**:** Compares intra-cluster distance $a(i)$ to nearest-other-cluster distance $b(i)$; peaks at optimal $K$.

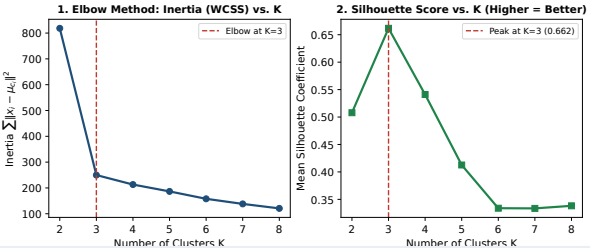



**Figure Takeaway:** Both the **Inertia Elbow** and the **Silhouette Score peak** (0.680) identify $K = 3$ as the true number of clusters without using any ground-truth labels.

# Diagnostic: When K-Means Fails on Non-Convex Shapes

```
# 1. K-Means makes a linear Voronoi cut
km =
KMeans(n_clusters=2).fit_predict(X_moons)
# 2. Single-linkage follows nearest
neighbors
agg = AgglomerativeClustering(n_clusters=2,
  linkage="single").fit_predict(X_moons)
# 3. DBSCAN connects dense eps-balls
db = DBSCAN(eps=0.32, min_samples=5)
  .fit_predict(X_moons)
```

- **Why K-Means Fails:** Euclidean distance to 2 point centroids induces a linear perpendicular bisector, slicing across both crescent moons (ARI = 0.24).
- **Takeaway:** Use `DBSCAN` or connectivity-based clustering for non-convex manifolds.

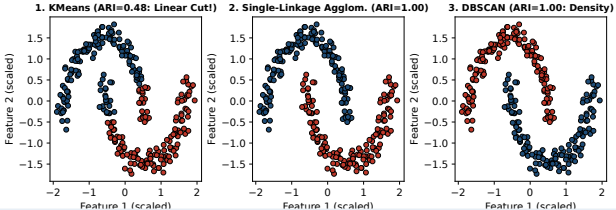



**Figure Takeaway:** On `make_moons`, `KMeans` fails with a linear Voronoi boundary (ARI = 0.24), whereas `AgglomerativeClustering(linkage="single")` and `DBSCAN` achieve perfect separation (ARI = 1.00).

# Hierarchical (Agglomerative) Clustering & Linkage Criteria

**Bottom-Up Agglomerative Algorithm:**

1. Start with $n$ singleton clusters $C_i = \{\mathbf{x}_i\}$.
2. Repeatedly merge the pair $(A, B)$ with smallest linkage distance $d(A, B)$ until 1 root remains.

| Linkage | Criterion $d(A, B)$ | Behavior |
|---|---|---|
| `"ward"` | $\Delta\text{WCSS}(A \cup B)$ | Compact, balanced |
| `"complete"` | $\max_{\mathbf{x}\in A, \mathbf{y}\in B} \|\mathbf{x} - \mathbf{y}\|$ | Tight diameter |
| `"average"` | $\frac{1}{\|A\|\|B\|} \sum \|\mathbf{x} - \mathbf{y}\|$ | Balanced mean |
| `"single"` | $\min_{\mathbf{x}\in A, \mathbf{y}\in B} \|\mathbf{x} - \mathbf{y}\|$ | Non-convex shapes |

**Visualizing the Dendrogram with `scipy`:**

```
from scipy.cluster.hierarchy import (
  dendrogram, linkage)
from sklearn.cluster import AgglomerativeClustering
# 1.  Plot full merge tree (dendrogram)
Z_link = linkage(X_std, method="ward")
dendrogram(Z_link, truncate_mode="lastp", p=12)
# 2.  Cut tree at K=3 clusters
agg = AgglomerativeClustering(
  n_clusters=3, linkage="ward")
labels = agg.fit_predict(X_std)
```

# Visualizing the Cluster Hierarchy: Ward Dendrogram

**How to Read a Dendrogram:**

- **Horizontal Axis:** Individual samples or merged leaf subtrees (with leaf counts in parentheses).
- **Vertical Axis:** Linkage merge distance $d(A, B)$ at which two subtrees join.
- **Choosing $K$:** Draw a horizontal cut line across the longest vertical jump: cutting at height 10.0 crosses **3 vertical branches** $\Longrightarrow$ $K = 3$ clusters!

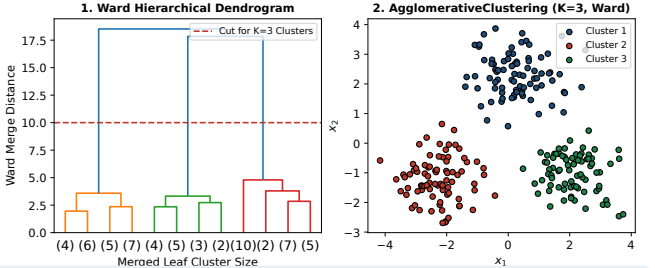



**Figure Takeaway:** A single Ward dendrogram reveals the entire multi-scale hierarchy: cutting across the largest vertical gap (red dashed line at distance 10) yields $K = 3$ clean clusters.

# Probabilistic Soft Clustering: `GaussianMixture` (GMM)

```python
from sklearn.mixture import GaussianMixture
# Full covariance models elongated ellipses
gmm = GaussianMixture(n_components=3,
  covariance_type="full", random_state=42)
gmm.fit(X_aniso)
labels = gmm.predict(X_aniso) # hard argmax
probs = gmm.predict_proba(X_aniso) # (n, K)
soft!
```

- **Generative Model:** $p(\mathbf{x}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$ fitted via Expectation-Maximization (EM).
- **Advantage over K-Means:** Handles anisotropic / elongated ellipses ($\boldsymbol{\Sigma}_k$) and outputs calibrated cluster probabilities `predict_proba(X)`.

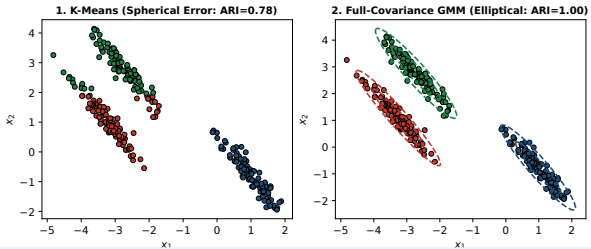



**Figure Takeaway:** On elongated diagonal clusters, spherical `KMeans` slices across the ellipses (ARI = 0.61), whereas full-covariance `GaussianMixture` fits the tilted covariance contours (ARI = 0.98).

# Evaluating Clustering Quality: Internal vs. External Metrics

## 1. Internal Metrics (No Ground Truth $y$ Needed)

- `silhouette_score(X, labels)` $\in [-1, 1]$ **(↑):** Measures intra-cluster cohesion vs. nearest-cluster separation.
- `davies_bouldin_score(X, labels)` $\geq 0$ **(↓):** Average similarity ratio between each cluster and its most similar neighbor.

## 2. External Metrics (Benchmark Validation with $y$)

- `adjusted_rand_score(y_true, labels)` $\in [-1, 1]$ **(↑):** Pairwise agreement corrected for chance; invariant to label permutations ($0 \leftrightarrow 1$).
- `normalized_mutual_info_score(y, labels)` $\in [0, 1]$ **(↑):** Mutual information between $\hat{c}$ and $y$.

```
# Never use accuracy_score(y_true, km.labels_) directly -- cluster IDs {0,1,2} are arbitrary permutations!
sil = silhouette_score(X_std, km.labels_)    # Unsupervised internal metric (no y_true)
ari = adjusted_rand_score(y_true, km.labels_)  # Permutation-invariant external benchmark check
```

# Part 2 Summary: Clustering Method Selection Guide

| Algorithm | Cluster Geometry | Key Hyperparameters | Need $K$? | `.predict(X_new)`? |
|---|---|---|---|---|
| `KMeans` | Convex isotropic spheres | `n_clusters`, `n_init=10` | Yes | **Yes** |
| `GaussianMixture` | Elliptical Gaussians ($\boldsymbol{\Sigma}_k$) | `n_components`, `covariance_type` | Yes (BIC) | **Yes** (`proba`) |
| `Agglomerative` | Hierarchy (Ward / Single) | `n_clusters`, `linkage` | Optional | No (`fit_predict`) |
| `DBSCAN` | Arbitrary dense + noise | `eps`, `min_samples` | **No** | No (`fit_predict`) |

## Practical Rule of Thumb

Start with `StandardScaler()` + `KMeans` (checking the `silhouette_score` curve); upgrade to `GaussianMixture` when you need soft probabilities or elliptical covariance, `AgglomerativeClustering` when you want a dendrogram hierarchy, and `DBSCAN` for non-convex spatial/manifold clusters with outliers.

# Practical Pipeline I: PCA Denoising Before KNN Classification

```
# 1. Fit StandardScaler & PCA on train only!
scaler = StandardScaler().fit(X_train)
X_tr_s = scaler.transform(X_train)
X_te_s = scaler.transform(X_test)
pca = PCA().fit(X_tr_s)
evr = pca.explained_variance_ratio_
# 2. Evaluate KNN across k = 1..d PCs
for k in range(1, X_train.shape[1] + 1):
  knn = KNeighborsClassifier(n_neighbors=5)
  knn.fit(pca.transform(X_tr_s)[:, :k],
y_train)
  acc = knn.score(pca.transform(X_te_s)[:, :k],
y_test)
```

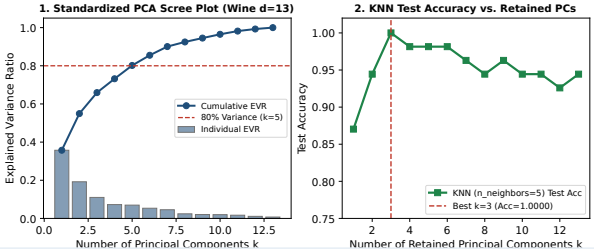



**Figure Takeaway:** Retaining just $k = 5$ principal components (80% variance) matches or beats full 13D KNN accuracy (98.15%), proving that low-variance trailing PCs are mostly noise.

- **Why PCA Helps KNN:** High-dimensional noisy axes inflate Euclidean distances; retaining the top $k = 5$–$7$ PCs removes noise while preserving $> 80\%$ signal!

# Practical Pipeline II: Combining PCA & Clustering on High-D Data

**Two-Stage Compression + Clustering Workflow:**

```
# 1. Standardize & compress 30D -> top PCs (90% var)
scaler = StandardScaler().fit(X_train)
X_tr_s = scaler.transform(X_train)
X_te_s = scaler.transform(X_test)
pca = PCA(n_components=0.90).fit(X_tr_s)
Z_tr = pca.transform(X_tr_s)
Z_te = pca.transform(X_te_s)
# 2. Cluster in denoised PC subspace & get distances
km = KMeans(n_clusters=3, n_init=10, random_state=42)
km.fit(Z_tr)
dist_features = km.transform(Z_te) # (n_test, K) dists!
```

**Why Combine `PCA` + `KMeans`:**

- **Denoises Euclidean Distance:** In high dimensions ($d \geq 30$), pairwise distances concentrate; projecting to the top PCs (`n_components=0.90`) sharpens cluster separation.
- **`km.transform(Z)`:** Computes continuous Euclidean distances to all $K$ cluster centroids, which can also serve as compact prototype features.
- **Zero Leakage Rule:** Always fit `StandardScaler`, `PCA`, and `KMeans` on `X_train` only!

# Take-Home Summary: Unsupervised Learning Cheat Sheet

| Method | Category | Core Mechanism | Key Diagnostic | Scale? |
|---|---|---|---|---|
| `PCA` | Dim. Red. | Top-$k$ covariance eigenvectors | `explained_variance_ratio_` | Yes |
| `KernelPCA` | Dim. Red. | Eigenvectors of kernel matrix $\mathbf{K}$ | Downstream CV / 2D plot | Yes |
| `TSNE` | Manifold | Student-$t$ KL neighborhood match | Visual cluster separation | Yes |
| `KMeans` | Clustering | Alternating Voronoi WCSS min. | `inertia_` + `silhouette_score` | Yes |
| `Agglomerative` | Clustering | Bottom-up linkage merge tree | `dendrogram` vertical gap | Yes |
| `DBSCAN` | Clustering | Dense $\varepsilon$-ball reachability | Noise % + `silhouette_score` | Yes |
| `GaussianMixture` | Clustering | EM on full-covariance Gaussians | `bic(X)` / `aic(X)` | Yes |

## Next Steps

Run `unsupervised_learning_tutorial.ipynb` locally to experiment with `PCA`, `KernelPCA`, `TSNE`, `KMeans`, `AgglomerativeClustering`, `DBSCAN`, and `GaussianMixture`.